UNIVERSITÉ DE LORRAINE

PROJET COMMUN Algorithmique et Complexité & ACL

Partie Algorithmique et Complexité : cahier des charges

On considère le problème qui consiste à ordonnancer un ensemble de N tâches indépendantes (i=1,2,3…N) sur m machines identiques. Chaque tâche i possède une durée opératoire p_i et une durée de latence q_i L'objectif est de trouver un ordonnancement permettant de minimiser le critère $L_{max} = \max_{i=1,2,\dots,N} \{C_i+q_i\}$ avec C_i la date de fin d'exécution de la tâche i. La préemption des tâches est interdite. Ce problème est noté P,m//L_{max}.

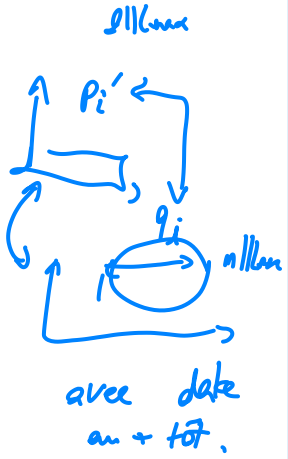
1/ Donner le ratio de performance garantie de l'algorithme qui consiste à ordonnancer les tâches dans l'ordre de Jackson (ordre décroissant des durées de latence) sur la machine disponible au plus tôt.

2/ Donner un algorithme de programmation dynamique basé sur la méthode de l'espace d'états. Donner une version de base de cet algorithme et expliquer comment améliorer la complexité ensuite tout en conservant l'optimalité de la solution.

3/ Donner un schéma d'approximation S (le meilleur possible) basé sur la technique de la modification des données en utilisant l'algorithme de la programmation dynamique précédent lorsque m est fixé (c'est-dire, considéré comme une constante).

Si m est devient variable (partie de l'entrée du problème) expliquer les difficultés de maintenir le schéma.

4/ La durée de latence q_i de chaque tâche i correspond en réalité à un temps de transport réalisé par un robot. Supposons que nous disposons uniquement de 2 robots pour réaliser les opérations de transport et que les N tâches sont déjà ordonnancées sur les m machines (selon la solution déjà donnée par le schéma d'approximation S), formuler le problème d'ordonnancement lié aux 2 robots et analyser la possibilité d'un algorithme d'approximation pour bien organiser les opérations de transport des robots.



5/ Programmez les algorithmes en C. Comparez les résultats de différentes méthodes sur des instances générées aléatoirement.

Délivrables : un rapport en PDF de 6 pages, une présentation en PDF et un code.

Date limite d'envoi : mardi 9 décembre 2025 à 12h00 (heure de Paris) par messagerie électronique à imed.kacem@univ-lorraine.fr et giorgio.lucarelli@univ-lorraine.fr

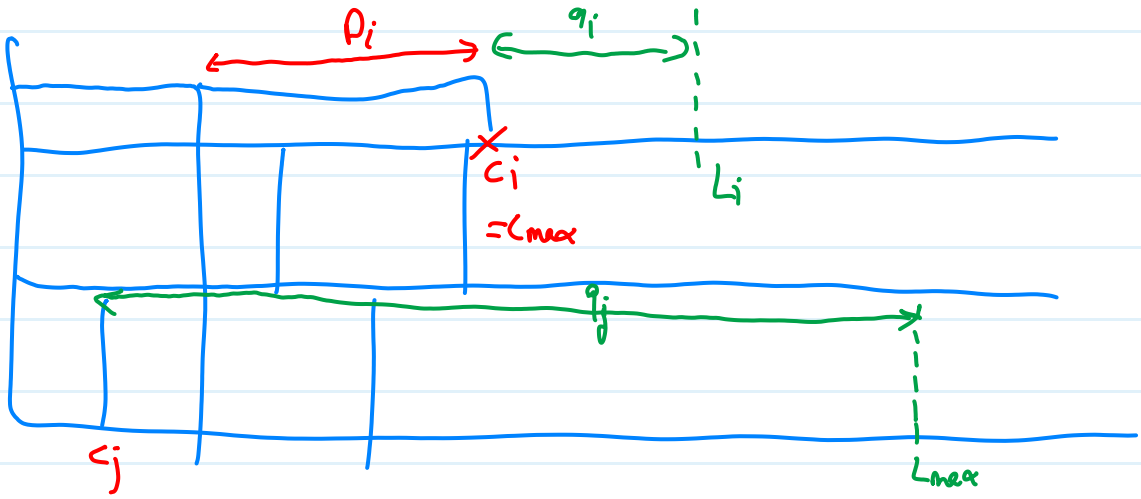
Date de la soutenance : le jeudi 11 décembre 2025. Un lien TEAMS sera envoyé au responsable du groupe.

le trigger le L_{max}

OPT ≥ $p_k + q_k$

$H = C_k + q_k = t + p_k + q_k$ parce que $C_k = t + p_k$

Problème : $L_i = C_i + q_i$

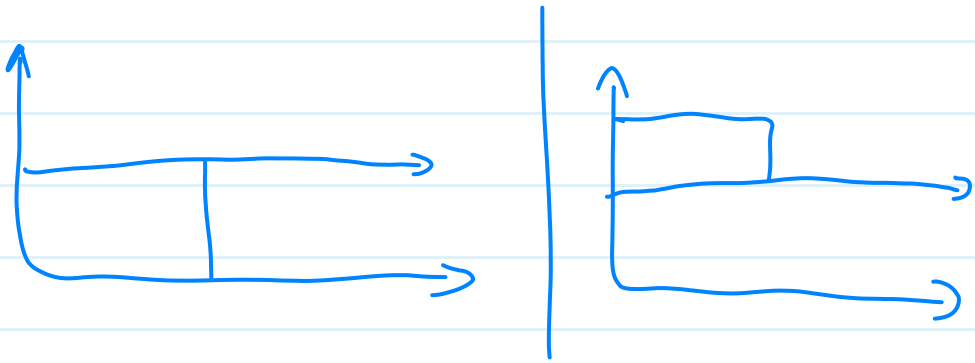


$L_{max} \neq C_{max} + q_{max}$

Pour $1 \| L_{max}$: On suppose l'ordre de Jackson

1 tâche :

$L_{max} = C_1 + q_1$

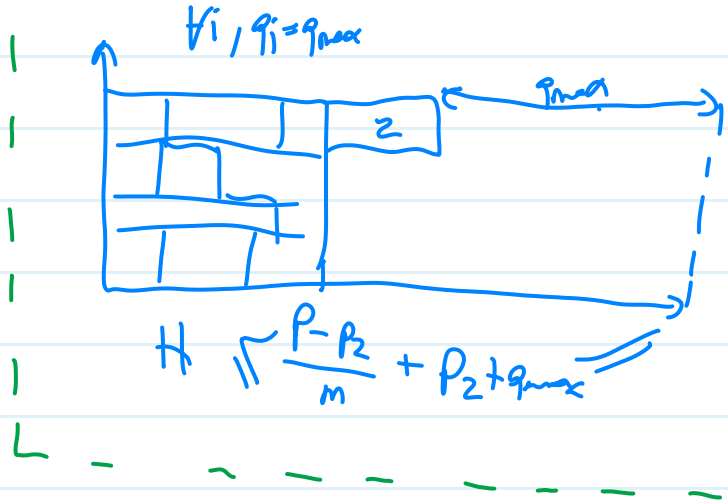


2 tâches :

$OPT \geq \max \left\{ \frac{P}{m}, q_{max} \right\}$

1/ Donner le ratio de performance garantie de l'algorithme qui consiste à ordonnancer les tâches dans l'ordre de Jackson (ordre décroissant des durées de latence) sur la machine disponible au plus tôt.

* $OPT \geq p_2$
* $OPT \geq q_{max}$
* $OPT \geq \frac{P}{m} + q_{min} \geq \frac{P}{m}$
* $OPT \leq P + q_{max}$
* $H \leq \frac{P - p_2}{m} + p_2 + q_{max} \leq$



* $H \leq \frac{P - p_2}{m} + p_2 + q_{max} \leq \frac{P - p_2}{m} + p_2 + OPT$
* $H - OPT \leq \frac{P - p_2}{m} + p_2 \leq \frac{P - p_2}{m} + p_2$

$$= \frac{P - p_2 + m p_2}{m}$$

$$= \frac{P + (m-1) p_2}{m}$$

$$\leq OPT + \frac{(m-1) p_2}{m}.$$

$$\leq OPT + (m-1) p_2$$

* $\frac{H - OPT}{OPT} \leq \frac{OPT + (m-1) p_2}{p_2} = \frac{OPT}{p_2} + m - 1$

2/ Donner un algorithme de programmation dynamique basé sur la méthode de l'espace d'états. Donner une version de base de cet algorithme et expliquer comment améliorer la complexité ensuite tout en conservant l'optimalité de la solution.

$$I = \langle N, M, p_1, p_2, \dots, p_n, q_1, q_2, \dots, q_n \rangle$$

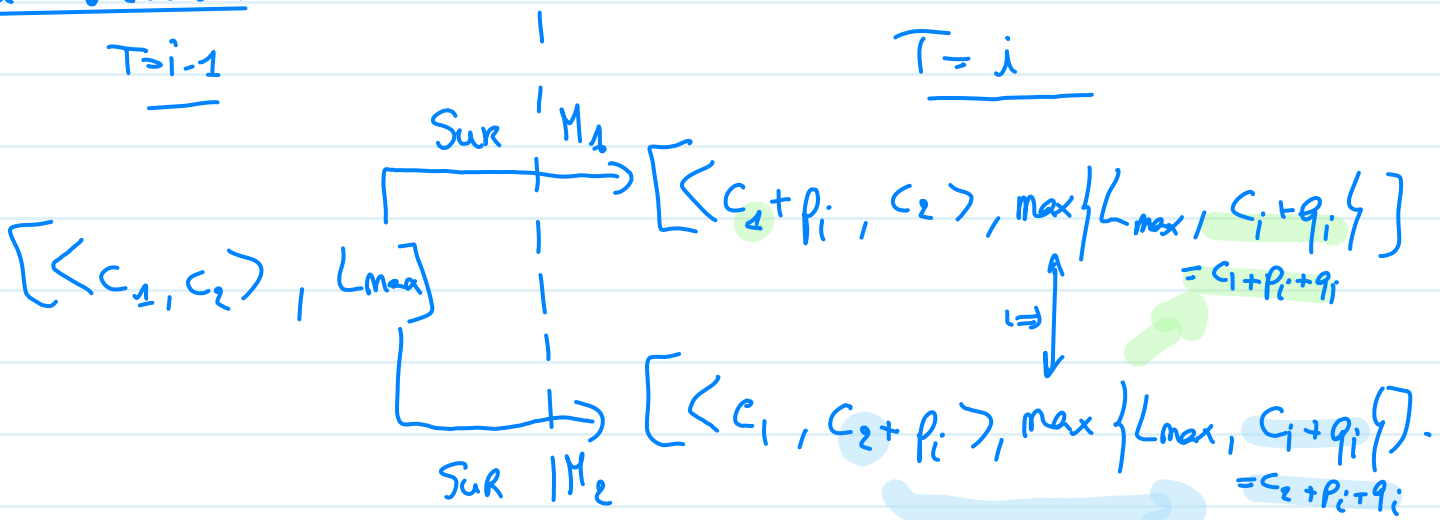
Décis° : Sur quelle machine je place la tâche i.

↳ On doit savoir quel est le C_{max} local à chaque machine. L'état est donc, pour $P||L_{max}$:

$$[\langle C_{max,M_1}, C_{max,M_2} \rangle, L_{max}]$$

* Le premier composant est un vecteur des C_{max} locaux à chaque machine
* Le 2e est le L_{max} global.

Règle d'évolut° :



L'algo pour 2 machines : Sans simplifi°

↓

$E_0 = \{[<0,0>, 0]\}$ — $O(1)$

On trie dès l'ordre de Jackson (décroissant des q_i). — $O(n \ell(n))$

Pour i de 1 à N faire

  $E_i = \{\}$

  p_i ;

  q_i ;

  Pour chaque état $[<c_1, c_2>, L_m]$ de E_{i-1} faire

    boucle de 1 à m pour manipuler les c_j

    Ajouter $[<c_1 + p_i, c_2>, \max\{L_m, c_1 + p_i + q_i\}]$ dans E_i

    Ajouter $[<c_1, c_2 + p_i>, \max\{L_m, c_2 + p_i + q_i\}]$ dans E_i

  fin

  ①

  Supprimer E_{i-1}

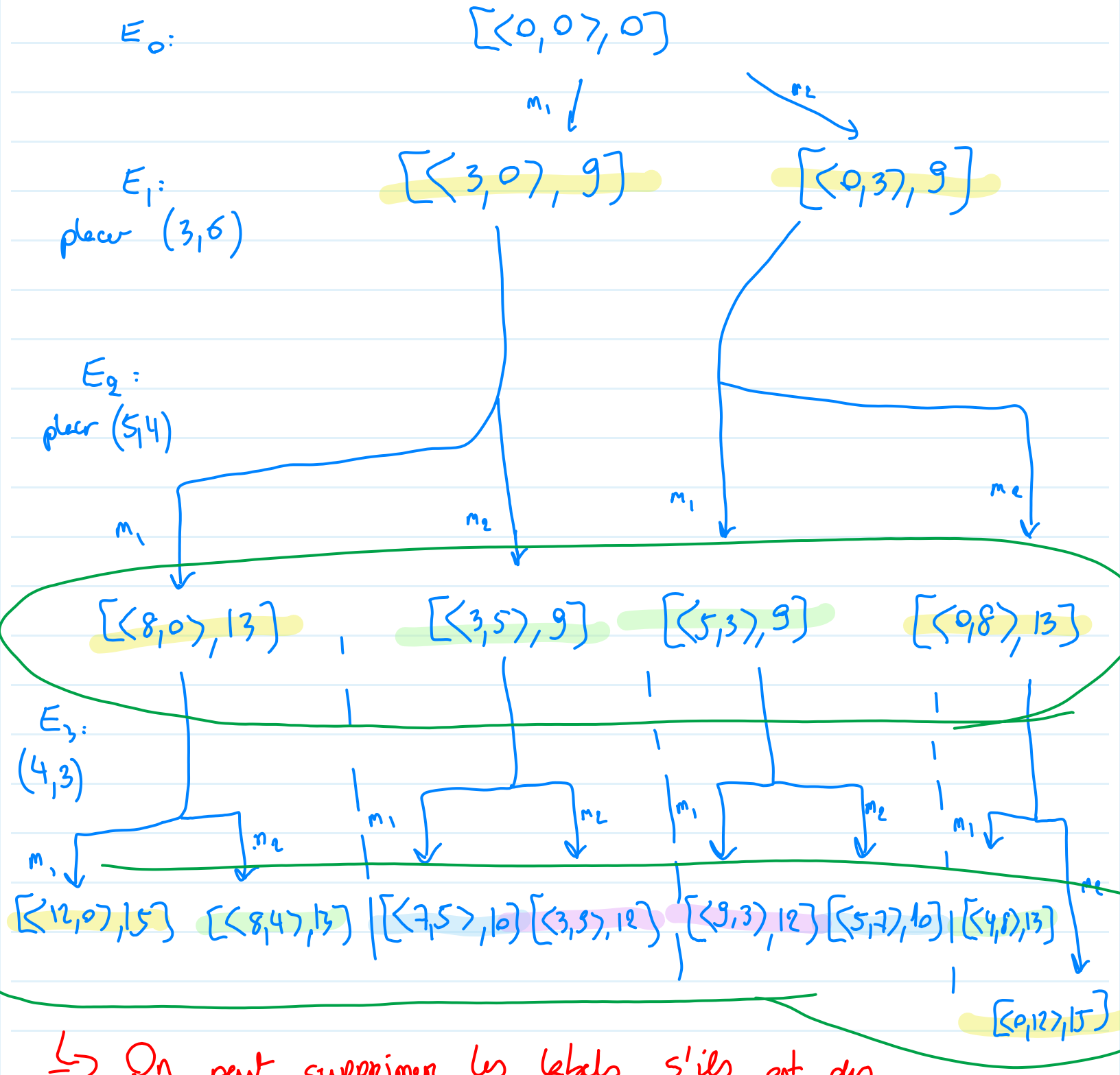
fin

Retourner $\min_{[\ldots, L_{max}] \in E_n} \{L_{max}\}$ — $O(\ldots)$

Ex :

i	1	2	3
p_i	3	5	4
q_i	6	4	3

Jackson



↳ On peut supprimer les labels s'ils ont des valeurs de vecteur similaires et si le L_{max} est similaire. ou ⊕ petit

3/ Donner un schéma d'approximation S (le meilleur possible) basé sur la technique de la modification des données en utilisant l'algorithme de la programmation dynamique précédent lorsque m est fixé (c'est-dire, considéré comme une constante).

On pose $I = \langle N, m, p_1, p_2, \dots, p_n, q_1, q_2, \dots, q_n \rangle$.

Le but est de minimiser $L_{max} = \max\{c_i + q_i\}$.
On opte pour la modificat° de l'instance tq:

$$\begin{cases} N^{\#} = N \\ m^{\#} = m \\ p_i^{\#} = \left\lfloor \frac{p_i}{\delta} \right\rfloor \cdot \delta \\ q_i^{\#} = \left\lfloor \frac{q_i}{\delta} \right\rfloor \cdot \delta \end{cases}$$

La valeur de δ est à déterminer.

Ainsi, $p_i^{\#} \leqslant p_i$ et $q_i^{\#} \leqslant q_i$ par arrondi inférieur.

On obtient $I^{\#} = \langle N^{\#}, m^{\#}, p_1^{\#}, p_2^{\#}, \dots, p_n^{\#}, q_1^{\#}, q_2^{\#}, \dots, q_n^{\#} \rangle$.

Analyse: L'objectif est de minimiser le L_{max}. $\min\{L_{max}\}$ soit $\min\{\max\{c_i + q_i\}\}$.

On note $L(A, B)$ la valeur du L_{max} depuis l'ordonnancement A appliqué à l'instance B.

$$L(A, B) = \max\{c_i^B + q_i^B\}$$

- $S^{\#}$ optimale de $I^{\#}$.
- S^{*} optimale de I.

On note : $p_i^{\#} \leqslant p_i$ et $q_i^{\#} \leqslant q_i$.

et comme $c_i = k + p_i$, on a $c_i^{\#} = k + p_i^{\#}$ $\forall k$,
$\Rightarrow c_i^{\#} \leqslant c_i$

Donc $c_i^{\#} + q_i^{\#} \leqslant c_i + q_i$

et $\max \{ c_i^{\#} + q_i^{\#} \} \leqslant \max \{ c_i + q_i \}$
soit $L_{max}^{\#} \leqslant L_{max}$.

De ce constat, on obtient la règle :

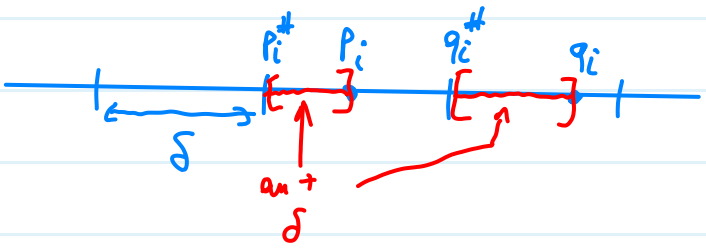
* (1) $L(A, I^{\#}) \leqslant L(A, I)$: $L_{max}^{\#} \leqslant L_{max}$ $\forall$ ordonnancement A.

Et aussi la règle de base :

* (2) $L(S^{\#}, I^{\#}) \leqslant L(A, I^{\#})$: $S^{\#}$ optimal pour $I^{\#}$

* (3) $L(S^{*}, I) \leqslant L(A, I)$: S^{*} optimal pour I.

On note aussi que :
Avec $p_i^{\#} \leqslant p_i$ et $q_i^{\#} \leqslant q_i$,



↓

* (4) $p_i - p_i^\# \leq \delta \Rightarrow p_i \leq \delta + p_i^\#$.
* (5) $q_i - q_i^\# \leq \delta \Rightarrow q_i \leq \delta + q_i^\#$

* (6) $L(A,B) = \max \{ c_i^B + q_i^B \}$

* (7) $c_i - c_i^\# \leq i\delta \Rightarrow c_i \leq i\delta + c_i^\# \Rightarrow c_i^\# \geq c_i - i\delta$

En appliquant $S^\#$ à I_1 on obtient:

$L(S^\#, I) \overset{(6)}{=} \max \{ c_i + q_i \}$

$\Rightarrow \overset{(7) \text{ et } (5)}{\leq} \max \{ c_i^\# + i\delta + q_i^\# + \delta \}$

$\Rightarrow \overset{i+1 \leq N+1}{=} \max \{ c_i^\# + q_i^\# + (i+1)\delta \}$

$\Rightarrow \leq \max \{ c_i^\# + q_i^\# + (N+1)\delta \}$ $\quad \max\{k_i + cst\} = \max\{k_i\} + cst$

$\Rightarrow = \max \{ c_i^\# + q_i^\# \} + (N+1)\delta$

$\Rightarrow$

$\Rightarrow \overset{(6)}{=} L(S^\#, I^\#) + (N+1)\delta$

$\Rightarrow \overset{(2)}{\leq} L(S^*, I^\#) + (N+1)\delta$

$\Rightarrow L(S^\#, I) \overset{(1)}{\leq} L(S^*, I) + (N+1)\delta$

On note $L(S^*, I) = OPT$:

$$L(S^\#, I) \leq OPT + (N+1)\delta.$$

On cherche $L(S^\#, I) \leq OPT + \varepsilon OPT$.

On veut donc: $(N+1)\delta \leq \varepsilon OPT$

$\Rightarrow \delta \leq \frac{\varepsilon OPT}{N+1}$ et comme $OPT \geq q_{max}$

$$\Rightarrow \delta = \frac{\varepsilon\, q_{max}}{N+1} \leqslant \frac{\varepsilon\, OPT}{N+1}.$$

donc: $L(S^{\#}, I) \leqslant OPT + (N+1)\delta$

$$= OPT + (N+1)\frac{\varepsilon\, q_{max}}{N+1}$$

$$= OPT + \varepsilon\, q_{max}$$

$q_{max} \leqslant OPT$ $\leqslant OPT + \varepsilon OPT.$

$$= (1+\varepsilon)\, OPT.$$

Donc $\delta = \dfrac{\varepsilon\, q_{max}}{N+1}$

et $q_{max}^{\#} = \left\lfloor \dfrac{q_{max}}{\delta} \right\rfloor \cdot \delta = \left\lfloor \dfrac{q_{max}(N+1)}{\varepsilon\, q_{max}} \right\rfloor \cdot \dfrac{\varepsilon\, q_{max}}{N+1}$

$$= \left(\frac{N+1}{\varepsilon} \right) \cdot \frac{\varepsilon\, q_{max}}{N+1}$$

↳) $q_{max}^{\#}$ peut prendre au plus $\frac{N+1}{\varepsilon}$ valeurs

donc: TODO estimer complexité de la boucle des contraintes selon $q_{max}^{\#}$.